

1. Osnove SQL (Stavek SELECT)

Uporablja se za branje podatkov iz baze.

- **Sintaksa:**

SQL

```
SELECT stolpec1, stolpec2 FROM tabela WHERE pogoj;
```

- **Ključni operatorji v WHERE:**

- `=`, `<>`, `<`, `>` (primerjava)
- `AND`, `OR`, `NOT` (logični operatorji)
- `LIKE '%tekst%'` (iskanje po delu besedila; `%` je poljubno število znakov)
- `IN ('A', 'B')` (preverjanje, če je vrednost v seznamu)
- `BETWEEN 10 AND 20` (vključno z mejama)

2. Združevanje podatkov (Aggregates & Grouping)

To uporabiš, ko želiš izračunati vsote, povprečja ali prešteti vrstice.

- **Funkcije:**

- `COUNT(*)` – prešteje vrstice.
- `SUM(stolpec)` – sešteje vrednosti.
- `AVG(stolpec)` – izračuna povprečje.
- **GROUP BY:** Če uporabiš agregatno funkcijo in želiš rezultat po kategorijah (npr. število študentov *po letnikih*), moraš te kategorije navesti v `GROUP BY`.
- **HAVING:** Je kot `WHERE`, ampak za rezultate agregatnih funkcij (npr. "pokaži oddelke, kjer je povprečna plača > 1000").

3. Stikanje tabel (JOIN)

Povezovanje podatkov iz dveh ali več tabel na podlagi skupnega ključa.

- **INNER JOIN:** Najpogostejši. Vrne le tisto, kar je v **obeh** tabelah (presečišče).
- **LEFT JOIN:** Vrne vse iz prve (leve) tabele, tudi če v desni ni ujemanja (vstavi `NULL`).
- **RIGHT JOIN:** Vrne vse iz desne tabele.
- **Primer:**

SQL

```
SELECT n.Ime, o.Naziv
FROM Zaposleni AS n
JOIN Oddelki AS o ON n.oddelek_id = o.id;
```

4. Ustvarjanje tabel (DDL - Data Definition Language)

Tukaj določiš strukturo in pravila (omejitve).

- **CREATE TABLE:**

SQL

```
CREATE TABLE Studenti (
    id_studenta INT PRIMARY KEY,      -- Unikatni identifikator
    ime VARCHAR(50) NOT NULL,          -- Ne sme biti prazno
    email VARCHAR(100) UNIQUE,         -- Ne sme se ponavljati
    starost INT CHECK (starost >= 18)  -- Pogoji za vnos
);
```

- **Tuji ključ (FOREIGN KEY):** Poveza na drugo tabelo, ki skrbi za integriteto (ne moreš vpisati študenta v oddelek, ki ne obstaja).

5. Spreminjanje podatkov (DML - Data Manipulation Language)

Ukazi za delo z vsebino vrstic.

- **INSERT:** Dodajanje novih vrstic.

SQL

```
INSERT INTO Studenti (id_studenta, ime) VALUES (1, 'Marko');
```

- **UPDATE:** Popravljanje obstoječih podatkov.

SQL

```
UPDATE Studenti SET ime = 'Luka' WHERE id_studenta = 1;
```

- **DELETE:** Brisanje vrstic.

SQL

```
DELETE FROM Studenti WHERE id_studenta = 1;
```

⚠ **Zlato pravilo:** Pri `UPDATE` in `DELETE` nikoli ne pozabi na `WHERE`, sicer boš pobrisal ali spremenil celo bazo!

Sintaksa se vedno začne z `CASE` in konča z `END`:

SQL

```
CASE
  WHEN pogoj THEN rezultat_1
  ELSE rezultat_2
END
```

```
import sqlite3

# 1. Povezava na bazo (ustvari datoteko, če ne obstaja)
conn = sqlite3.connect('moja_baza.db')
cursor = conn.cursor()

# 2. Ustvarjanje tabele
cursor.execute('''CREATE TABLE IF NOT EXISTS Uporabniki
(id INTEGER PRIMARY KEY, ime TEXT)''')

# 3. Vstavljanje podatkov (uporablaj ? za varnost pred SQL injection!)
ime_uporabnika = "Luka"
cursor.execute("INSERT INTO Uporabniki (ime) VALUES (?)", (ime_uporabnika,))

# 4. Shranjevanje sprememb (OBVEZNO pri INSERT/UPDATE/DELETE)
conn.commit()

# 5. Branje podatkov
cursor.execute("SELECT * FROM Uporabniki")
vsi = cursor.fetchall()
for vrstica in vsi:
    print(vrstica)

# 6. Zapiranje povezave
conn.close()
```